Agent Phantom Recovery

13-api-specification.md

Production API Specification

Version: 1.0

Part 1

---

1. Purpose

This document defines the public and internal API contracts for Agent Phantom Recovery.

The API layer serves as the communication boundary between:

```
Antigravity IDE
        ↓
API Layer
        ↓
Backend Services
```

and between:

```
Orchestrator
Planner
Reasoner
Verifier
Memory
Tools
Repository Services
```

The API architecture is designed to support:

- Long-horizon execution
- Multi-agent workflows
- Realtime collaboration
- Tool orchestration
- Repository intelligence
- Memory retrieval
- Verification pipelines
- Future model flexibility

2. API Design Philosophy

2.1 Capability-Driven APIs

APIs should expose capabilities rather than model vendors.

Correct:

```
/agents/planner
/agents/reasoner
/agents/verifier
```

Avoid:

```
/llm/gemini
/llm/openai
/llm/anthropic
```

This prevents vendor lock-in.

2.2 Provider Independence

The API contract must remain unchanged if:

```
Planner
↓
Gemini
```

becomes:

```
Planner
↓
Claude
```

or

```
Planner
↓
OpenAI
```

Backend adapters should absorb provider changes.

2.3 Task-Centric Design

Everything revolves around tasks.

Primary resource:

```
Task
```

Secondary resources:

```
Plan
Memory
Evidence
Finding
Patch
Verification
Report
```

2.4 Realtime First

The API must support:

- Streaming
- Live updates
- Progress notifications
- Browser synchronization
- Terminal synchronization

because Antigravity is an execution workspace rather than a static web application.

2.5 Versioned APIs

All APIs must be versioned.

Example:

```
/api/v1/tasks
```

Future:

```
/api/v2/tasks
```

No breaking changes should occur inside a version.

---

3. API Standards

Base URL

```
/api/v1
```

Content Type

Request:

```
Content-Type: application/json
```

Response:

```
Content-Type: application/json
```

Timestamp Standard

All timestamps:

```
ISO 8601 UTC
```

Example:

```
{
  "created_at": "2027-01-01T10:00:00Z"
}
```

ID Standard

All resources:

```
UUID v4
```

Example:

```
7cb3c5d7-4b95-4d9f-a0b4-8d04f2d09c4d
```

---

4. Authentication Architecture

Authentication Flow

```
User
 ↓
Login
 ↓
JWT Access Token
 ↓
Authenticated Requests
```

Supported Authentication Methods

Email / Password

OAuth (Future)

Enterprise SSO (Future)

---

5. Authentication APIs

Login

Endpoint

```
POST /api/v1/auth/login
```

Request

```
{
  "email": "user@example.com",
```

```
  "password": "********"
}
```

Response

```
{
  "access_token": "",
  "refresh_token": "",
  "expires_in": 3600
}
```

Refresh Token

Endpoint

```
POST /api/v1/auth/refresh
```

Request

```
{
  "refresh_token": ""
}
```

Response

```
{
  "access_token": "",
  "expires_in": 3600
}
```

Logout

Endpoint

```
POST /api/v1/auth/logout
```

Response

```
{
  "success": true
}
```

Current User

Endpoint

```
GET /api/v1/auth/me
```

Response

```
{
  "id": "",
  "email": "",
  "username": "",
  "role": ""
}
```

---

6. Project APIs

Projects are top-level workspaces.

Create Project

Endpoint

```
POST /api/v1/projects
```

Request

```
{
  "name": "Project Name",
  "description": "Description"
}
```

Response

```
{
  "id": "",
  "name": "",
  "description": ""
}
```

List Projects

Endpoint

```
GET /api/v1/projects
```

Response

```
{
  "projects": []
}
```

Get Project

Endpoint

```
GET /api/v1/projects/{projectId}
```

Response

```
{
  "id": "",
  "name": "",
  "description": "",
  "created_at": ""
}
```

Update Project

Endpoint

```
PATCH /api/v1/projects/{projectId}
```

Delete Project

Endpoint

```
DELETE /api/v1/projects/{projectId}
```

---

7. Repository APIs

Repositories are attached to projects.

Register Repository

Endpoint

```
POST /api/v1/repositories
```

Request

```
{
  "project_id": "",
  "source_url": "",
  "branch": "main"
}
```

Response

```
{
  "repository_id": "",
  "status": "REGISTERED"
}
```

Start Repository Ingestion

Endpoint

```
POST /api/v1/repositories/{repositoryId}/ingest
```

Response

```
{
  "job_id": "",
  "status": "STARTED"
}
```

Repository Status

Endpoint

```
GET /api/v1/repositories/{repositoryId}/status
```

Response

```
{
  "repository_id": "",
  "status": "INDEXING",
  "progress": 62
}
```

Repository Summary

Endpoint

```
GET /api/v1/repositories/{repositoryId}/summary
```

Response

```
{
  "language": "Python",
  "framework": "FastAPI",
  "files": 183,
  "symbols": 4127,
```

```
  "dependencies": 96
}
```

Repository Structure

Endpoint

```
GET /api/v1/repositories/{repositoryId}/structure
```

Response

```
{
  "directories": [],
  "files": []
}
```

Repository Search

Endpoint

```
POST /api/v1/repositories/{repositoryId}/search
```

Request

```
{
  "query": "authentication middleware"
}
```

Response

```
{
  "results": []
}
```

Repository Call Graph

Endpoint

```
GET /api/v1/repositories/{repositoryId}/call-graph
```

Response

```
{
  "nodes": [],
  "edges": []
}
```

Repository Dependency Graph

Endpoint

```
GET /api/v1/repositories/{repositoryId}/dependency-graph
```

Response

```
{
  "packages": [],
  "relationships": []
}
```

---

8. Pagination Standard

List endpoints should support:

```
?page=1
&page_size=20
```

Response Format

```
{
  "items": [],
  "page": 1,
```

```
  "page_size": 20,
  "total": 100
}
```

---

9. Standard Success Response

```
{
  "success": true,
  "data": {}
}
```

---

10. Standard Error Response

```
{
  "success": false,
  "error": {
    "code": "RESOURCE_NOT_FOUND",
    "message": "Repository not found"
  }
}
```

---

11. Security Requirements

Every endpoint must support:

- JWT validation
- Authorization checks
- Audit logging
- Request tracing

Sensitive endpoints require ownership validation.

---

12. Antigravity IDE Integration Requirements

The API layer must support:

Workspace Initialization

```
Project
Repository
Task
Memory
```

loading in a single request cycle.

Browser Workspace

Repository APIs must provide browser context data.

Source Code Workspace

Repository APIs must provide:

- file trees
- search
- code retrieval

Task Workspace

Task APIs must support live status retrieval.

Timeline Workspace

All actions must be traceable through events.

End of API Specification Part 1

Next Section:

Part 2

- Task APIs
- Plan APIs
- Memory APIs
- Agent APIs
- Planner APIs
- Reasoner APIs
- Verifier APIs

These APIs form the execution core of Agent Phantom Recovery.